DATABRICKS

Data Engineer Associate Exam

Chapter 3 — Comprehensive Study Guide

Data Engineering in Databricks: Lakeflow & Spark Declarative Pipelines

Based on official Databricks course slides & documentation • June 2026

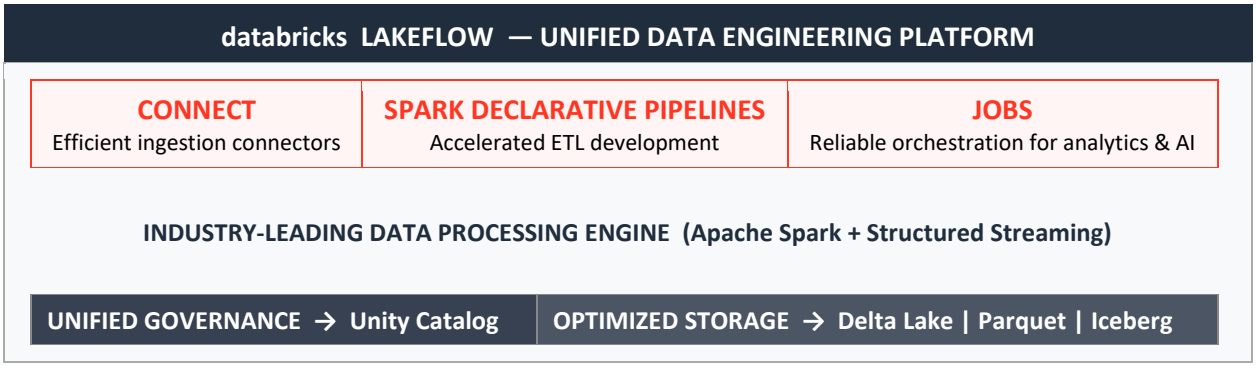
Table of Contents

Table of Contents 2
1. The Databricks Lakeflow Architecture 4
1.1 The Three Lakeflow Components 4
2. Spark Declarative Pipelines vs. Jobs 5
2.1 How to Create Each (from the UI) 5
Creating a Spark Declarative Pipeline (ETL Pipeline) 5
Creating a Job Task 5
3. Unity Catalog, the Databricks UI & Storage Layers 6
3.1 The Three-Level Namespace 6
3.2 What is a Volume? 6
4. Message Queues & Their Role in Databricks 7
4.1 Common Message Queue Systems 7
4.2 Role in a Databricks Pipeline 7
5. The Medallion Architecture: Bronze, Silver, Gold 8
5.1 Bronze Layer — Raw Data Ingestion 8
5.2 Silver Layer — Cleaned & Transformed Data 8
5.3 Gold Layer — Business-Level Aggregations 8
6. Dataset Types in Spark Declarative Pipelines 10
6.1 Streaming Table (ST) 10
6.2 Materialized View (MV) 11
6.3 Views and Temporary Views 11
Temporary View 11
View (Persistent) 11
6.4 Syntax Name Changes: DLT → Lakeflow 12
7. Key SQL Commands & Code Snippets 13
7.1 Creating a Bronze Streaming Table from JSON Files 13
7.2 Creating a Silver Streaming Table (Transformation) 13
7.3 Creating a Gold Materialized View (Aggregation) 14
7.4 Pipeline Configuration Parameters 14
8. Incremental Processing & Idempotency 15
8.1 What is Idempotency? 15
8.2 How Incremental Processing Works Across Two Runs 15
9. Common Pipeline Settings 16
9.1 Compute 16
9.2 Code Assets 16

9.3 Configuration (Parameters) 16
9.4 Additional Pipeline Settings 17
10. Compute Types in Databricks & Cost Reduction 18
10.1 Key Cost-Reduction Features 18
Autoscaling 18
Serverless Compute 18
Photon Query Engine 18
Incremental Refresh for Materialized Views 18
11. Ensure Data Quality with Expectations 20
11.1 Syntax 20
11.2 The Three Violation Actions 20
11.3 Complete Example with All Three Constraint Types 20
12. Streaming Joins Overview 22
12.1 Stream-to-Static Join (Stream-Snapshot Join) 22
12.2 Stream-to-Stream Join (via Materialized View) 22
12.3 Stream-Stream Incremental Join (Advanced — Out of Scope) 22
13. Change Data Capture (CDC) & Slowly Changing Dimensions 24
13.1 What is Change Data Capture? 24
13.2 Slowly Changing Dimensions (SCD) 24
SCD Type 1 — Overwrite (No History) 24
SCD Type 2 — Historical Versioning (Outside Course Scope) 24
13.3 AUTO CDC INTO — The Lakeflow CDC Command 25
14. Deploying a Pipeline to Production 27
14.1 Scheduling the Pipeline 27
14.2 Email Notifications 27
14.3 Monitoring with the Pipeline Event Log 27
14.4 Accessing the Event Log 28
15. Additional Advanced Features 29
16. Exam Quick-Reference Cheat Sheet 30
16.1 Key Terms in Plain English 30
16.2 SQL Command Reference Card 30
16.3 Critical Exam Watch-Outs 31
16.4 Official Databricks Documentation Links 32

1. The Databricks Lakeflow Architecture

The central theme of Chapter 3 is **Lakeflow** — Databricks' end-to-end unified data engineering solution. Lakeflow sits above a best-in-class storage and governance layer and gives data engineers a single environment for ingestion, transformation, and orchestration. The diagram below (from Slide 2) shows how all pieces relate:



1.1 The Three Lakeflow Components

Lakeflow Connect is a collection of efficient, pre-built ingestion connectors that simplify pulling data into Databricks from popular enterprise applications (Workday, Salesforce), databases (SQL Server, PostgreSQL), cloud object storage (S3, ADLS, GCS), and streaming message buses (Kafka, Pub/Sub, Kinesis). It eliminates the need for custom ETL code just to land data.

Lakeflow Spark Declarative Pipelines (SDP) (previously called Delta Live Tables / DLT) is a framework for building batch and streaming data pipelines using SQL or Python. You declare what you want the data to look like and Databricks handles the execution plan, dependency resolution, error handling, checkpointing, and retries automatically.

Lakeflow Jobs (previously called Workflows) is a workflow automation and orchestration tool. It coordinates multiple tasks — notebooks, pipelines, Python scripts, SQL queries, dbt tasks, Spark JARs — within complex, repeatable workflows. Jobs handle scheduling, dependency sequencing between tasks, and integration with CI/CD.

EXAM NOTE

The course (and exam) focuses specifically on Lakeflow Spark Declarative Pipelines — the ETL component.

📖 **Official Docs:** https://docs.databricks.com/aws/en/delta-live-tables/index.html

2. Spark Declarative Pipelines vs. Jobs

This distinction is one of the most frequently tested topics on the exam. The key insight is that these two tools solve different problems — one defines what data to compute; the other controls when and how to run everything.

Spark Declarative Pipelines — WHAT to compute	Lakeflow Jobs — WHEN & HOW to run things
Declare transformation logic in SQL or Python	Orchestrates tasks: notebooks, pipelines, scripts
System auto-infers execution order (DAG)	You define task order and dependencies manually
Handles checkpointing, retries, error recovery	Handles scheduling, triggers, and notifications
Designed for ETL: Bronze → Silver → Gold	Coordinates multiple systems and workloads
Idempotent by design — safe to re-run	Can trigger a Declarative Pipeline as a task
Built-in data quality with Expectations	Tracks run history, logs, and retry policies
Supports both batch and streaming	CI/CD-friendly via Databricks Asset Bundles

2.1 How to Create Each (from the UI)

Creating a Spark Declarative Pipeline (ETL Pipeline)

There are two equivalent paths in the Databricks UI (Slide 16):

1. **From the workspace:** Click the + (options) button in the navigation bar → Create → ETL Pipeline
2. **From Jobs & Pipelines:** Navigate to Jobs & Pipelines in the main nav → click Create → select ETL Pipeline

Both paths open the multi-file Pipeline Editor where you write SQL or Python files defining your dataset transformations.

Creating a Job Task

Navigate to Jobs & Pipelines → Create Job → Add Task. Task types include: Notebook, ETL Pipeline, dbt, Python script, SQL query, Spark JAR. A single Job can chain many tasks with defined dependencies between them.

3. Unity Catalog, the Databricks UI & Storage Layers

Unity Catalog (UC) is Databricks' centralised data governance layer. Think of it as a combination phone book and security guard — it tracks every data asset across workspaces and controls who can access what. It provides access control, auditing, data lineage, quality monitoring, and data discovery.

3.1 The Three-Level Namespace

Every object in Unity Catalog has a fully-qualified name with three parts:

```
catalog.schema.table

Example:  1_bronze_db.default.orders_bronze
```

Object	Equivalent Concept	Example
Catalog	Top-level namespace (like a database server)	1_bronze_db
Schema	Groups tables within a catalog (like a database)	default
Table	Stores data rows (managed or external Delta table)	orders_bronze
View	Virtual table — no physical storage	orders_view
Volume	Governed storage for non-tabular files (JSON, CSV, images)	/Volumes/dbacademy/ops/orders/

3.2 What is a Volume?

A **Volume** in Unity Catalog is a governed location for non-tabular files. Unlike tables which store structured row/column data, Volumes store raw files such as JSON, CSV, Parquet, images, or binary data — all under the Unity Catalog access control model. In Spark Declarative Pipelines, source files referenced in `read_files()` calls commonly point to a Volume path.

When to use a Volume vs. a Table: Use a Volume when you want governed access to raw/semi-structured files that are not yet ingested into a Delta table. Once ingested and structured, the data becomes a Delta table (e.g., a Bronze streaming table).

📖 Unity Catalog docs | Volumes docs

4. Message Queues & Their Role in Databricks

A message queue is a system that acts like a post office for data. Producers send messages (events, records) into the queue, and consumers read from it asynchronously, at their own pace. This architecture decouples the data producer (an application generating events) from the data consumer (your Databricks pipeline).

4.1 Common Message Queue Systems

System	Provider	Use Case
Apache Kafka	Self-hosted / Confluent Cloud	High-throughput event streaming, log aggregation
Google Pub/Sub	Google Cloud	GCP-native event delivery and fan-out
Amazon Kinesis	AWS	Real-time streaming data ingestion on AWS
Azure Event Hubs	Microsoft Azure	Azure-native streaming and CDC integration

4.2 Role in a Databricks Pipeline

Message queues are one of the primary streaming data sources that Lakeflow Connect ingests from (Slide 10). They appear at the very beginning of the pipeline, feeding into the Bronze Streaming Table. Use a message queue when:

- Data arrives continuously in real time (clickstream events, IoT sensor readings, financial transactions)
- You need near-real-time processing — new records should be available in your pipeline within seconds or minutes
- You want to decouple your upstream applications from your data platform (producers write to Kafka; pipelines read from Kafka independently)

In a Spark Declarative Pipeline, the FROM clause would reference the message queue using a Structured Streaming reader. For example, reading from a Kafka topic feeds directly into the Bronze layer Streaming Table.

📖 Kafka connector docs

5. The Medallion Architecture: Bronze, Silver, Gold

The Medallion Architecture is the foundational data organisation pattern used across Databricks. It divides data processing into three progressive quality layers, with data flowing from raw ingestion all the way to business-ready outputs (Slides 11–15).



5.1 Bronze Layer — Raw Data Ingestion

The Bronze layer is the landing zone for data exactly as it arrives from source systems. No transformation of raw values occurs here. Additional metadata columns are often appended to provide context about the ingestion process (e.g., the file name the record came from, the exact ingestion timestamp).

- **Typical dataset type:** Streaming Table
- **Data quality:** Raw — may contain nulls, duplicates, malformed records
- **Source:** Cloud storage files (JSON, CSV, Parquet), message queues, databases via Lakeflow Connect
- **Example table:** `orders_bronze`, `status_bronze`, `customers_bronze`

5.2 Silver Layer — Cleaned & Transformed Data

The Silver layer is where data cleaning, transformation, filtering, type casting, deduplication, and joins happen. It turns the raw Bronze data into a usable, well-structured table that operational queries and downstream aggregations can rely on.

- **Typical dataset type:** Streaming Table
- **Key activities:** Type casting, column renaming, filtering invalid rows, normalisation, applying data quality Expectations
- **Example table:** `orders_silver`, `status_silver`, `customers_bronze_clean`

5.3 Gold Layer — Business-Level Aggregations

The Gold layer contains the final, business-ready data products. These are pre-aggregated, pre-joined, and optimised for specific downstream consumers such as BI dashboards, machine learning models, streaming analytics systems, and executive reporting.

- **Typical dataset type:** Materialized View
- **Key activities:** GROUP BY aggregations, complex joins across Silver tables, filtered views for specific business use cases
- **Example table:** `gold_orders_by_date`, `full_order_info_gold`, `cancelled_orders`, `delivered_orders`

📖 Medallion Architecture docs

6. Dataset Types in Spark Declarative Pipelines

Within a Spark Declarative Pipeline, there are three dataset types you define using SQL (or Python). Understanding their differences — how they process data, when to use each, and their limitations — is essential for the exam (Slides 22–34).

Feature	Streaming Table (ST)	Materialized View (MV)	Temporary View / View
Stores physical data	✅ Yes	✅ Yes	❌ No — virtual only
Processes new data only	✅ Yes (incremental)	✅ Incremental on Serverless	N/A
Registered in Unity Catalog	✅ Yes	✅ Yes	Temp: ❌ | View: ✅
Requires STREAM keyword	✅ Required for ST sources	❌ No	❌ No
Can be streaming source	✅ Yes	❌ No	❌ No
Typical pipeline layer	Bronze & Silver	Gold (and any layer)	Intermediate logic only
SQL creation command	CREATE OR REFRESH STREAMING TABLE	CREATE OR REFRESH MATERIALIZED VIEW	CREATE [TEMPORARY] VIEW

6.1 Streaming Table (ST)

A Streaming Table is a Delta table with built-in support for **incremental, exactly-once data processing**. Each time the pipeline runs, only data that has not yet been processed is ingested and appended to the table. This is achieved through Auto Loader checkpoints that record which files or offsets have already been consumed.

- **Exactly-once guarantee:** Each file or message is guaranteed to be processed exactly once — no duplicates, no missed data.
- **Auto Loader integration:** When reading files from cloud storage, Lakeflow uses Auto Loader internally to detect new files incrementally and checkpoint progress.
- **Batch and streaming:** Streaming Tables can be populated in either batch mode (triggered runs) or streaming mode (continuous runs) — you choose at the pipeline level.
- **Always append:** New processed data is always appended to the Streaming Table; the table grows over time.

📌 EXAM NOTE

In SQL, you MUST use the STREAM keyword when reading from a Streaming Table as a source. Without it, Databricks performs a full batch read rather than an incremental read.

Deprecated DLT equivalent: CREATE OR REFRESH STREAMING LIVE TABLE (old syntax still works for compatibility)

📖 Streaming Table docs

6.2 Materialized View (MV)

A Materialized View is a **pre-computed query result stored physically as a Delta table**. Unlike a regular view (which recalculates on every query), an MV is computed once by the pipeline and updated automatically whenever upstream data changes. This makes queries against Gold-layer aggregations extremely fast.

- **Auto-maintained by the pipeline:** You do not manage the refresh schedule manually — the pipeline handles it.
- **Incremental refresh (Serverless only):** A cost-based optimiser decides whether to do a full recompute or only recompute the changed portions. This avoids expensive full recomputation on every pipeline run.
- **No STREAM keyword:** MVs reference source streaming tables without the STREAM keyword — the MV runtime tracks change detection itself.
- **Can appear anywhere:** While commonly used at the Gold layer, MVs can be used at any layer whenever you need a pre-computed query result.

⚠ WATCH OUT

As of Q2 2025: Materialized views that use Expectations (data quality constraints) are ALWAYS fully refreshed on each pipeline run, even on Serverless compute.

Deprecated DLT equivalent: CREATE OR REFRESH LIVE TABLE

📖 Incremental refresh for Materialized Views

6.3 Views and Temporary Views

Views create virtual tables based on a SQL query — they store no physical data. There are two types used in Declarative Pipelines:

Temporary View

- **Scope:** Exists only for the duration of a single pipeline run. Not persisted after the run completes.
- **Visibility:** Private to the defining pipeline — not registered in Unity Catalog. Other pipelines, notebooks, and users cannot see it.
- **Best for:** Intermediate query logic that supports your pipeline but does not need to be exposed to downstream consumers.
- Syntax: `CREATE TEMPORARY VIEW view_name AS SELECT ...`

View (Persistent)

- **Scope:** Registered in Unity Catalog — visible and queryable by other users and pipelines.

- **No physical storage:** Query is re-executed each time someone queries the view.
- **Limitations:** The pipeline must be a Unity Catalog pipeline. Views cannot contain streaming queries and cannot be used as a streaming source in another pipeline.
- Syntax: `CREATE VIEW view_name AS SELECT ...`

6.4 Syntax Name Changes: DLT → Lakeflow

Existing DLT users should note that the command names have been updated. The old syntax is still supported for backward compatibility.

New Lakeflow Syntax	Old DLT Syntax (Deprecated)
CREATE OR REFRESH STREAMING TABLE	CREATE OR REFRESH STREAMING LIVE TABLE
CREATE OR REFRESH MATERIALIZED VIEW	CREATE OR REFRESH LIVE TABLE
CREATE VIEW	CREATE LIVE VIEW
CREATE TEMPORARY VIEW	CREATE TEMPORARY LIVE VIEW

7. Key SQL Commands & Code Snippets

7.1 Creating a Bronze Streaming Table from JSON Files

The following code (from Slide 24) creates a Bronze streaming table called orders_bronze by reading JSON files from a Volume path using Auto Loader:

```
CREATE OR REFRESH STREAMING TABLE 1_bronze_db.orders_bronze AS
SELECT
    *,
    current_timestamp() AS processing_time,
    _metadata.file_name  AS source_file
FROM STREAM read_files(
    '/Volumes/dbacademy/ops/labuser/orders',
    format => 'JSON'
);
```

Clause / Function	What It Does
CREATE OR REFRESH STREAMING TABLE	Creates the table if it does not exist; refreshes it on subsequent runs without creating duplicates
1_bronze_db.orders_bronze	Fully-qualified table name: schema.table_name in the default catalog
current_timestamp()	Adds the ingestion timestamp as a metadata column for auditing
_metadata.file_name	Auto Loader system column — captures the source filename for each ingested row
FROM STREAM read_files(...)	STREAM keyword enables incremental Auto Loader reads; read_files() scans the path and returns file contents in tabular form
format => 'JSON'	Tells read_files() to parse each file as JSON

7.2 Creating a Silver Streaming Table (Transformation)

The Silver table transforms data from the Bronze streaming table. Only new rows arriving in Bronze since the last pipeline run are processed (Slide 25):

```
CREATE OR REFRESH STREAMING TABLE 2_silver_db.orders_silver AS
SELECT
    order_id,
    timestamp(order_timestamp) AS order_timestamp,
    customer_id,
    notifications
FROM STREAM 1_bronze_db.orders_bronze;

-- The STREAM keyword tells the pipeline to read only NEW rows
-- from orders_bronze that have not yet been processed.
```

EXAM NOTE

The STREAM keyword in the FROM clause is what makes this incremental. Without it, every pipeline run would reprocess all Bronze data — defeating the purpose of streaming tables.

7.3 Creating a Gold Materialized View (Aggregation)

The Gold Materialized View aggregates data from the Silver streaming table. Note the absence of the STREAM keyword (Slide 29):

```
CREATE OR REFRESH MATERIALIZED VIEW 3_gold_db.gold_orders_by_date AS
SELECT
    date(order_timestamp) AS order_date,
    count(*)              AS total_daily_orders
FROM 2_silver_db.orders_silver      -- No STREAM keyword!
GROUP BY date(order_timestamp);

-- Incremental refresh is used where possible (Serverless compute).
-- The cost-based optimizer decides: full refresh vs. partial refresh.
```

7.4 Pipeline Configuration Parameters

Rather than hardcoding paths and values in SQL, use pipeline configuration parameters (key-value pairs). Reference them in SQL with the ${key-name} syntax (Slide 45):

```
-- In Pipeline Settings: set  source = /Volumes/dbacademy/ops/dbbar

CREATE OR REFRESH STREAMING TABLE bronze AS
SELECT *
FROM STREAM read_files(
    '${source}/orders',      -- ${source} is replaced at runtime
    format => 'JSON'
);
```

TIP

Using configuration parameters makes the same pipeline code reusable across dev, staging, and production environments — just change the parameter values, not the code.

8. Incremental Processing & Idempotency

8.1 What is Idempotency?

Idempotency means that running the same operation once or a hundred times produces the same result — there are no duplicates, no missed data. Spark Declarative Pipelines are designed to be idempotent because:

- Streaming Tables use Auto Loader checkpoints to track what has already been processed.
- Each file/message is guaranteed to be processed exactly once (exactly-once semantics).
- The CREATE OR REFRESH syntax means re-running the pipeline updates the existing table rather than creating a duplicate.

8.2 How Incremental Processing Works Across Two Runs

The following walkthrough (from Slides 14–15) shows how data flows through the pipeline on first and subsequent runs:

Run	What Happens at Bronze	What Happens at Silver	What Happens at Gold
Run 1 (first execution)	All files from source are read and appended to the Bronze streaming table for the first time.	All rows in Bronze are transformed and appended to Silver (first population).	Gold Materialized View is computed from the full Silver table.
Run 2 (new files arrive)	Auto Loader detects only the NEW files (checkpoint tracks what was already read). Only new rows are appended to Bronze.	Only the new rows from Bronze (since last run) are transformed and appended to Silver. Previous rows are untouched.	Gold MV is updated — incremental refresh or full refresh depending on the optimizer.

TIP

This incremental approach is what makes Spark Declarative Pipelines efficient at scale. You never pay compute cost to reprocess data you have already handled.

9. Common Pipeline Settings

Spark Declarative Pipelines offer a variety of configuration settings accessible from the Pipeline Settings panel (click the gear icon in the multi-file editor). The three most important settings covered in Chapter 3 (Slides 41–47) are Compute, Code Assets, and Configuration.

9.1 Compute

The Compute setting determines the execution environment for your pipeline. There are three options:

Compute Type	Description	Recommended?
Serverless	Fully managed by Databricks. No cluster provisioning needed. Auto-scales automatically. Enables incremental refresh for Materialized Views. Optional Serverless Performance Optimized setting for time-sensitive workloads.	✅ YES — recommended for all new pipelines
Classic (Fixed Size)	A fixed-size cluster you configure manually. Users need permission to create compute. Workspace admins can configure cluster policies to control access.	Use for specific requirements
Classic with Autoscaling	Enhanced autoscaling is enabled by default for all new Classic pipelines. Automatically adjusts cluster size based on workload volume, optimising resource utilisation and cost.	Good alternative to Serverless

9.2 Code Assets

Controls which code files are included in the pipeline execution:

- **Pipeline Root Folder:** Automatically includes all relevant files (.sql , .py , notebooks) within the folder for your pipeline project. This can be a Git folder, enabling version control and team collaboration.
- **Source Code Section:** Lets you specify individual subfolders or individual files to include — useful when you only want a subset of files in your root folder to be part of the pipeline.

📌 EXAM NOTE

Recommended: Write pipeline logic in .sql or .py files (not notebooks). The multi-file editor provides better tooling — interactive DAG, data previews, dry-run validation, and execution insights.

9.3 Configuration (Parameters)

A pipeline's Configuration is a map of key-value pairs that parameterise your SQL and Python code. These are set in the Pipeline Settings UI and referenced in code using the ${key-name} syntax.

- Improve code readability and maintainability by centralising key values
- Enable code reuse across different environments (dev, staging, prod) without changing the code itself

- Common parameters: source file path, target schema name, date range filters

9.4 Additional Pipeline Settings

Setting	Purpose
Default Location for Data Assets	Sets the default catalog and schema where pipeline outputs (tables, views) are stored
Environment	A pip-style requirements list for Python package dependencies needed by your pipeline code
Tags	Organise and categorise pipelines for easy management, cost allocation, and discovery
Budget	Define cost controls and spending limits for pipeline execution
Advanced Settings	Fine-tuning options including publishing the event log as a Delta table for external querying

10. Compute Types in Databricks & Cost Reduction

Understanding the different compute types and how they affect cost is a key exam topic. Databricks offers several compute types, each suited to different workloads.

Compute Type	What It Is	Cost Behaviour
Serverless (Pipelines)	Fully managed by Databricks. No infrastructure provisioning. Auto-scales. Pay only during active execution.	Most cost-efficient for pipelines — no idle cost
All-Purpose Cluster	Long-running interactive cluster for notebooks and exploratory analysis. Stays running until manually stopped.	Can be expensive — always running; terminate when not in use
Job Cluster	Spins up specifically for a Job run, then terminates automatically when the job completes.	Better than all-purpose — pay only during job execution
SQL Warehouse	Optimised serverless engine for SQL queries and BI dashboards. Auto-suspends when idle.	Cost-efficient — auto-suspend reduces idle time
Instance Pool	Pre-provisioned VMs shared across multiple clusters. Reduces cluster startup time.	Reduces startup cost; machines held warm reduce cold-start delay

10.1 Key Cost-Reduction Features

Autoscaling

Automatically adds or removes worker nodes from a Classic cluster based on current workload volume. Enabled by default for all new Classic Compute Declarative Pipelines. Prevents paying for idle workers during low-load periods.

Serverless Compute

The recommended choice for all new Declarative Pipelines. No cluster management overhead — Databricks provisions and manages compute automatically. You pay only for the actual compute used during pipeline execution. Additional benefits include incremental refresh for Materialized Views.

Photon Query Engine

Photon is a next-generation vectorised query engine written in C++ that runs alongside Apache Spark on Databricks clusters. It significantly accelerates SQL workloads and Delta Lake operations, which means you process more data per unit of time, reducing the overall cost-per-query.

Incremental Refresh for Materialized Views

On Serverless compute, the cost-based optimiser decides whether to incrementally update a Materialized View or fully recompute it. Incremental refresh avoids rebuilding the entire view when only a small portion of upstream data has changed.

📖 Photon docs | Serverless pipeline docs

11. Ensure Data Quality with Expectations

Expectations are data quality constraints applied to rows as they flow through the pipeline. They validate data row-by-row during ETL and allow you to decide what happens when a row fails the check (Slides 49–52).

11.1 Syntax

```
CONSTRAINT constraint_name EXPECT (column_condition) [ON VIOLATION action]

-- constraint_name : a descriptive label for the rule
-- column_condition: a SQL boolean expression
-- action          : WARN (default), DROP ROW, or FAIL UPDATE
```

11.2 The Three Violation Actions

Action	What Happens to the Row	What Happens to the Pipeline	When to Use
WARN (default)	Row IS written to the target table — the invalid data still lands	Pipeline continues normally. Invalid row counts are logged in metrics alongside valid row counts.	When you want visibility into data quality issues without blocking the pipeline. Good for monitoring.
DROP ROW ON VIOLATION DROP ROW	Row is EXCLUDED from the output — silently removed	Pipeline continues. The count of dropped rows is logged alongside other metrics.	When invalid rows are unusable and should not pollute downstream tables, but the pipeline can continue without them.
FAIL UPDATE ON VIOLATION FAIL UPDATE	Row triggers a FAILURE of the specific flow	Only that flow fails. Other independent flows in the same pipeline continue running. Manual intervention is required to fix the issue.	When a data quality violation is critical and must be resolved before any downstream processing should proceed.

11.3 Complete Example with All Three Constraint Types

The following example (from Slide 51) creates an orders_silver streaming table with three different expectation types applied simultaneously:

```
CREATE OR REFRESH STREAMING TABLE 2_silver_db.orders_silver
(
  -- WARN: logs rows where notifications is not 'Y' or 'N', but still writes them
  CONSTRAINT valid_notifications
    EXPECT (notifications IN ('Y','N')),

  -- FAIL: if order_timestamp is before 2021-01-01, stop this flow entirely
  CONSTRAINT valid_date
    EXPECT (order_timestamp > '2021-01-01')
```

```
    ON VIOLATION FAIL UPDATE,

  -- DROP: silently remove rows where customer_id is NULL
  CONSTRAINT valid_id
    EXPECT (customer_id IS NOT NULL)
    ON VIOLATION DROP ROW
)
AS
SELECT
    order_id,
    timestamp(order_timestamp) AS order_timestamp,
    customer_id,
    notifications
FROM STREAM 1_bronze_db.orders_bronze;
```

⚠ WATCH OUT

As of Q2 2025: Materialized Views that use Expectations are ALWAYS fully refreshed on each pipeline run, even on Serverless compute where incremental refresh would normally apply.

📖 Expectations docs

12. Streaming Joins Overview

When joining tables inside Declarative Pipelines, the join behaviour depends on the types of tables involved. There are three patterns covered in Chapter 3 (Slides 54–60):

12.1 Stream-to-Static Join (Stream-Snapshot Join)

This is the simplest join pattern: a **Streaming Table joined to a static (non-streaming) reference table**. The streaming table changes incrementally, while the static table is a stable lookup (e.g., a country code mapping, a product catalogue, or a user roles table).

- **How it works:** As new rows arrive in the streaming source, each new row is joined against ALL rows in the static lookup table in real time.
- **Efficiency:** Only the new (streaming) rows are evaluated in each pipeline run — the static table does not need to be reprocessed.
- **Result type:** Produces a new Streaming Table. New enriched rows are appended on each run.
- **Use case:** Enriching streaming transaction records with country names, product descriptions, or region codes from a static reference table.

📖 Streaming joins docs

12.2 Stream-to-Stream Join (via Materialized View)

When **both sides of a join are Streaming Tables**, a Materialized View is required to handle the join efficiently. The MV computes the join across all current data from both streaming tables each time the pipeline runs.

- **How it works:** As new data is appended to either (or both) streaming tables, the Materialized View detects the changes and recomputes the join across all current rows.
- **Incremental refresh:** The MV uses incremental refresh where possible to avoid full recomputation, making this efficient even at scale.
- **Use case:** Combining a customer activity stream with a product catalogue update stream to produce a continuously updated enriched view.

12.3 Stream-Stream Incremental Join (Advanced — Out of Scope)

A stream-stream incremental join processes only the NEW incoming data from two streaming sources — past data is not reconsidered. This pattern requires windowing logic, watermarking, and stateful stream processing.

📌 EXAM NOTE

Stream-stream incremental joins are outside the scope of this course and exam. They are covered in the 'Databricks Streaming and DLT Advanced' course.

13. Change Data Capture (CDC) & Slowly Changing Dimensions

13.1 What is Change Data Capture?

Change Data Capture (CDC) is a technique for tracking and capturing data changes — inserts, updates, and deletes — from a source system, and then applying those changes to a target table in your Lakehouse.

Simple analogy: Instead of copying an entire customer database every night, CDC tracks only what changed (new customer added, address updated, account deleted) and applies just those changes to your copy. This is far more efficient than full reloads.

A CDC source table typically contains rows representing change events, each with an operation type column (INSERT, UPDATE, DELETE) and a sequence column that establishes the chronological order of events.

13.2 Slowly Changing Dimensions (SCD)

SCDs define how historical changes are stored in the target table. There are two main types:

SCD Type 1 — Overwrite (No History)

When a record is updated, the existing row in the target table is simply overwritten with the new values. When a record is deleted, the row is removed from the target. No history of previous values is kept — only the current state of each record is retained.

- **Use when:** Only the most recent data matters. Examples: current product price, current customer email address, current employee status.

BEFORE (customers target table)

ID	Name	Address
1	Peter	1 Blue Rd
2	Samarth	22 Front St

⟹

AFTER SCD Type 1 applied

ID	Name	Address
1	Peter	**123 Main St ✓**
3	Kostas	**100 Athens Dr ✓**

Peter updated | Samarth deleted | Kostas added

SCD Type 2 — Historical Versioning (Outside Course Scope)

When a record updates, the old record is preserved in the table (marked as inactive) and a new record is inserted (marked as active). This maintains a full history of every change.

- **__START_AT column:** Timestamp when this version of the row became active.
- **__END_AT column:** Timestamp when this version became inactive. A NULL value means the row is CURRENTLY ACTIVE.
- **Use when:** Historical data is important — e.g., tracking every address a customer has ever had, audit trails, regulatory compliance.

13.3 AUTO CDC INTO — The Lakeflow CDC Command

Lakeflow provides a built-in declarative CDC mechanism using the AUTO CDC INTO statement (formerly APPLY CHANGES INTO in DLT). This handles inserts, updates, and deletes automatically without writing complex MERGE INTO SQL.

```
-- Step 1: Create the target streaming table
CREATE OR REFRESH STREAMING TABLE customers;

-- Step 2: Define the CDC flow using AUTO CDC INTO
CREATE FLOW scd_type_1_flow AS
AUTO CDC INTO customers              -- Target: apply changes to this table
FROM STREAM updates                  -- Source: streaming table of change events
KEYS (CustomerID)                    -- Primary key to match source and target
rows
APPLY AS DELETE WHEN operation = 'DELETE'  -- Treat rows with operation=DELETE as
deletes
SEQUENCE BY ProcessDate              -- Column that orders events chronologically
COLUMNS * EXCEPT (operation)        -- Include all columns except the operation
indicator
STORED AS SCD TYPE 1;               -- SCD Type 1 = overwrite (default if omitted)

-- For SCD Type 2 historical tracking, use: STORED AS SCD TYPE 2
```

Clause	What It Does
AUTO CDC INTO customers	Apply all CDC events (inserts, updates, deletes) to the target streaming table named 'customers'
FROM STREAM updates	Source streaming table containing the change event records
KEYS (CustomerID)	The column(s) that uniquely identify each row — used to match source change events to target rows
APPLY AS DELETE WHEN operation = 'DELETE'	Instructs the pipeline which column value signals a delete event (rather than an insert or update)
SEQUENCE BY ProcessDate	Establishes chronological order for applying changes — critical when multiple changes exist for the same key
COLUMNS * EXCEPT (operation)	Selects all columns for the target except the operation indicator column
STORED AS SCD TYPE 1	Default: overwrite mode. Use SCD TYPE 2 for historical versioning. If omitted, defaults to SCD Type 1.

⚠ WATCH OUT

AUTO CDC INTO requires the target table to already exist as a STREAMING TABLE (created separately with CREATE OR REFRESH STREAMING TABLE). You do not write a SELECT statement for the target — the CDC flow populates it.

📖 AUTO CDC INTO docs

14. Deploying a Pipeline to Production

Once you have built and tested your Spark Declarative Pipeline in development, three key operational tasks make it production-ready: scheduling, email notifications, and monitoring via the Event Log (Slides 62–71).

14.1 Scheduling the Pipeline

A production pipeline should run on an automated schedule. Databricks provides two scheduling modes:

Triggered Mode	Continuous Mode
Can be triggered manually or set to run on a recurring schedule (e.g., every 15 minutes, hourly, daily)	Pipeline runs indefinitely, continuously processing new data as it arrives
Refreshes selected tables using data available at the start of execution	Keeps Streaming Tables and Materialized Views up to date in near real-time
Stops automatically once all updates are complete	Monitors source dependencies and only triggers updates when source data changes
Ideal for batch use cases with defined data freshness SLAs	Best for streaming use cases where data latency and freshness are critical
More cost-efficient: compute runs only during execution	Higher cost: compute runs continuously

14.2 Email Notifications

As part of scheduling, you can configure email alerts to notify stakeholders automatically. You choose which pipeline events trigger a notification:

- When the pipeline starts
- When the pipeline succeeds
- When the pipeline fails
- Any combination of the above

This is especially useful in production so that on-call engineers are immediately alerted to failures without needing to constantly check the UI.

14.3 Monitoring with the Pipeline Event Log

The Pipeline Event Log is the central monitoring tool for Declarative Pipelines. It captures all critical runtime information in one place (Slide 69):

Event Log Category	What It Contains
Audit Logs	Tracks who did what and when within the pipeline — useful for compliance and accountability

Event Log Category	What It Contains
Data Quality Checks	Results of all Expectations (constraint pass/fail counts, rows dropped, warnings triggered)
Pipeline Progress	Status and progress of each dataset update (pending, running, completed, failed)
Data Lineage	How data flows and transforms through each stage of the pipeline — useful for impact analysis

14.4 Accessing the Event Log

There are two ways to access the Event Log:

3. **Default (Hidden Delta Table):** The event log is automatically written as a hidden Delta table in the pipeline's default catalog and schema. Access it programmatically via the documentation.
4. **Published (Public Delta Table):** Enable 'Publish event log to metastore' in Advanced Settings to make the event log queryable like any other table:

```
-- Query the published event log table
SELECT * FROM catalog.schema.event_log_table_name
ORDER BY timestamp DESC;

-- Filter for data quality failures only
SELECT * FROM catalog.schema.event_log_table_name
WHERE event_type = 'flow_progress'
  AND details:flow_progress:data_quality:dropped_records > 0;
```

📖 Pipeline event log docs

15. Additional Advanced Features

These features extend pipeline capabilities beyond the core course content. Know them at a high level for the exam.

Feature	Description	Official Docs
Multi-Flow Pipelines	Add multiple flows that write to the SAME target table, creating a union of different data sources into one output table. Useful for merging two customer systems into a single unified customers table.	docs.databricks.com → flows
Sinks (External Output)	The Sink API lets you write pipeline outputs to external systems beyond Delta tables: Apache Kafka, Delta (cross-workspace), Azure Event Hubs. Great for real-time downstream consumption.	docs.databricks.com → sinks
Liquid Clustering	Automatically optimises Delta file layout for query performance during pipeline runs. No manual OPTIMIZE commands needed.	Included with Delta Lake support
Row-Level Security & Column Masking	Enforce fine-grained access control directly within Streaming Tables and Materialized Views using Unity Catalog policies.	Unity Catalog docs
Change Data Feed (CDF)	Captures row-level changes from Streaming Tables and makes them available for downstream CDC propagation or audit logging.	Delta Lake CDF docs
Databricks Asset Bundles (DABs)	Programmatically validate, deploy, and run Databricks resources (including pipelines) for CI/CD workflows. DABs are the production deployment mechanism for teams practicing DevOps.	docs.databricks.com/bundles
Photon Engine	Next-generation C++ vectorised query engine. Significantly accelerates SQL and Delta operations. Reduces cost-per-query by processing more data per unit time.	docs.databricks.com/photon

16. Exam Quick-Reference Cheat Sheet

16.1 Key Terms in Plain English

Term	Plain-English Explanation
Idempotency	Run it once or a hundred times — same result every time. No duplicates, no missed data.
Declarative	You say WHAT you want the data to look like; the system figures out HOW to produce it.
Incremental Processing	Only process new data that has arrived since the last run. Skip what you've already handled.
Auto Loader	Databricks tool that automatically detects and ingests new files from cloud storage paths as they arrive.
Checkpoint	A saved record of which data has already been processed — ensures exactly-once guarantees on re-runs.
DAG (Directed Acyclic Graph)	A visual map of how pipeline datasets depend on each other. Databricks infers this automatically — code order does not matter.
ETL	Extract, Transform, Load. The process of moving and reshaping data from source systems into a target store.
CDC (Change Data Capture)	Tracking inserts, updates, and deletes in a source system and applying them to a target table.
SCD Type 1	Overwrite strategy — only keep current data, no history of previous values.
SCD Type 2	History strategy — preserve old records alongside new ones using timestamp columns (__START_AT, __END_AT).
Expectation	A named data quality rule (CONSTRAINT) applied to rows flowing through a pipeline.
Serverless	Databricks manages all infrastructure automatically. You just write code and pay for actual usage.
Photon	Faster Spark query engine written in C++. Accelerates SQL and Delta Lake operations.
Delta Lake	Open-source storage format with ACID transactions, time travel, schema enforcement, and streaming support.
Volume	Unity Catalog-governed storage location for non-tabular files (JSON, CSV, images, etc.).

16.2 SQL Command Reference Card

Command	Purpose
CREATE OR REFRESH STREAMING TABLE schema.table AS SELECT ... FROM STREAM source	Create or refresh a Streaming Table (Bronze or Silver layer). The STREAM keyword enables incremental, exactly-once reads.

Command	Purpose
FROM STREAM read_files('/path', format => 'JSON')	Read files incrementally using Auto Loader. New files only on each run.
CREATE OR REFRESH MATERIALIZED VIEW schema.view AS SELECT ... FROM source	Create or refresh a Materialized View (Gold layer). No STREAM keyword — MV manages its own refresh.
CREATE TEMPORARY VIEW name AS SELECT ...	Create a view that exists only during the pipeline run. Not registered in Unity Catalog.
CREATE VIEW name AS SELECT ...	Create a Unity Catalog-registered virtual view. No physical data stored.
CONSTRAINT name EXPECT (condition)	Add a WARN data quality rule. Invalid rows are still written; counts are logged.
CONSTRAINT name EXPECT (condition) ON VIOLATION DROP ROW	Drop rows that fail the quality check. Pipeline continues.
CONSTRAINT name EXPECT (condition) ON VIOLATION FAIL UPDATE	Fail the specific flow if any row violates the constraint.
AUTO CDC INTO target FROM STREAM source KEYS (col) SEQUENCE BY col ...	Apply CDC changes (inserts/updates/deletes) from source to target streaming table.
STORED AS SCD TYPE 1	Overwrite mode in AUTO CDC INTO. Default if omitted.
STORED AS SCD TYPE 2	Historical tracking mode in AUTO CDC INTO. Preserves old records with __START_AT / __END_AT columns.
SELECT * FROM catalog.schema.event_log_table	Query the published pipeline event log Delta table for monitoring and auditing.

16.3 Critical Exam Watch-Outs

CRITICAL TRAPS — Don't get caught!

5. **STREAM keyword:** Required when reading from a Streaming Table as a streaming source. Without it, you get a full batch read — no incremental processing.
6. **Materialized Views — no STREAM:** MVs reference their source tables WITHOUT the STREAM keyword. The MV runtime manages change tracking.
7. **TEMPORARY VIEW is NOT in Unity Catalog:** Use CREATE VIEW (not TEMPORARY VIEW) if you need the view visible and queryable by other users or pipelines.
8. **AUTO CDC INTO — target must exist first:** You must CREATE OR REFRESH STREAMING TABLE the target before the AUTO CDC INTO statement. There is no SELECT inside the target creation.
9. **Default SCD type:** If STORED AS is omitted from AUTO CDC INTO, it defaults to SCD TYPE 1 (overwrite). Always specify explicitly.
10. **Code order does not matter:** Pipeline dataset dependencies are inferred automatically from the DAG. You can write SQL files in any order.
11. **Serverless is recommended:** Databricks recommends Serverless compute for ALL new Declarative Pipelines. It also enables incremental refresh for MVs.
12. **Autoscaling default:** Enhanced autoscaling is enabled by DEFAULT for all new pipelines using Classic compute.

13. **Expectations + MV (2025Q2 update):** Materialized views that use Expectations are ALWAYS fully refreshed — incremental refresh does not apply.
14. **View limitations:** Persistent VIEWs require a Unity Catalog pipeline, cannot contain streaming queries, and cannot be used as streaming sources.

16.4 Official Databricks Documentation Links

Topic	URL
Lakeflow / Delta Live Tables overview	https://docs.databricks.com/aws/en/delta-live-tables/index.html
Load data with Lakeflow Declarative Pipelines	https://docs.databricks.com/aws/en/delta-live-tables/load.html
Incremental refresh for Materialized Views	https://docs.databricks.com/aws/en/delta-live-tables/materialized-views.html
CREATE TEMPORARY VIEW / CREATE VIEW	https://docs.databricks.com/aws/en/delta-live-tables/views.html
Data quality Expectations	https://docs.databricks.com/aws/en/delta-live-tables/expectations.html
AUTO CDC INTO	https://docs.databricks.com/aws/en/delta-live-tables/cdc.html
Configure a Serverless pipeline	https://docs.databricks.com/aws/en/delta-live-tables/serverless.html
Develop & debug with Pipelines Editor	https://docs.databricks.com/aws/en/delta-live-tables/develop-pipelines.html
Pipeline event log	https://docs.databricks.com/aws/en/delta-live-tables/observability.html
Streaming joins	https://docs.databricks.com/aws/en/delta-live-tables/streaming.html
Sinks (external output)	https://docs.databricks.com/gcp/en/delta-live-tables/sinks.html
Flows (multi-source → one target)	https://docs.databricks.com/gcp/en/delta-live-tables/flows.html
Unity Catalog overview	https://docs.databricks.com/aws/en/data-governance/unity-catalog/index.html
Volumes	https://docs.databricks.com/aws/en/connect/unity-catalog/volumes.html
Medallion Architecture	https://docs.databricks.com/aws/en/lakehouse/medallion.html
Databricks Asset Bundles (DABs)	https://docs.databricks.com/aws/en/dev-tools/bundles/index.html
Photon query engine	https://docs.databricks.com/aws/en/compute/photon.html
Jobs / Workflows	https://docs.databricks.com/aws/en/jobs/index.html
Kafka connector	https://docs.databricks.com/aws/en/structured-streaming/kafka.html

Study guide compiled from Databricks Chapter 3 course slides:

"Introduction to Data Engineering in Databricks" and "Lakeflow Spark Declarative Pipeline Fundamentals"
Generated June 2026 • Databricks Data Engineer Associate Exam Preparation